

Όραση Υπολογιστών

1η Σειρά Εργαστηριακών Ασκήσεων

Ραφτόπουλος Μιχάλης - AM: 03120114

Εαρινό Εξάμηνο 2024

Μέρος 1: Ανίχνευση Ακμών σε Γκρίζες Εικόνες

1.1 Δημιουργία Εικόνων Εισόδου

1.1.1 Διαβάζουμε την εικόνα εισόδου και την μετατρέπουμε από BGR σε Grayscale.

Listing 1: Import image and convert from BGR to GRAY

```
1 # Load and display first image
2 image0 = cv2.imread('edgetest_24.png')
3
4 # Convert the image from BGR to GRAY
5 image0 = cv2.cvtColor(image0, cv2.COLOR_BGR2GRAY)
6
```

1.1.2 Προκειμένου να βρεθούν οι κατάλληλες τυπικές αποκλίσεις σ_n με ακρίβεια ώστε να έχουμε τα ζητούμενα PSNR, αλλά και για ευκολότερη μελλοντική χρήση, υλοποιήθηκε η συνάρτηση PSNRnoise(). Η συνάρτηση αυτή δέχεται μια εικόνα και μια τιμή PSNR και της εφαρμόζει λευκό γκαουσιανό θόρυβο, με την κατάλληλη τυπική απόκλιση σ_n . Για τον υπολογισμό αυτό χρησιμοποιείται ένας απλός αλγόριθμος δυαδικής αναζήτησης.

Listing 2: implementation of PSNRnoise() function

```
1 def PSNRnoise(image, psnr):
2     # Gets image and PSNR as input,
3     # returns image with gaussian noise, so that it has the given PSNR
4
5     # Add Gaussian noise and tweak standard deviation (binary search) until the PSNR has the
6     right value
7     mean = 0
8     std_dev = 20
9     min_std_dev = None
10    max_std_dev = None
11    psnr_value = None
12
13    while psnr_value != psnr:
14        gaussian_noise = np.random.normal(mean, std_dev, image0.shape).astype('uint8')
15        noisy_image = cv2.add(image, gaussian_noise)
16
17        # Calculate PSNR
18        psnr_value = 20 * np.log10((noisy_image.max() - noisy_image.min()) / std_dev)
19
20        if psnr_value > psnr:
21            if max_std_dev:
22                temp = std_dev
23                std_dev += (max_std_dev - std_dev)/2
24                min_std_dev = temp
25            else:
26                min_std_dev = std_dev
27                std_dev *= 2
```

```

27         elif psnr_value < psnr:
28             if min_std_dev:
29                 temp = std_dev
30                 std_dev -= (std_dev - min_std_dev)/2
31                 max_std_dev = temp
32             else:
33                 max_std_dev = std_dev
34                 std_dev/=2
35
36         return noisy_image, psnr_value, std_dev
37

```

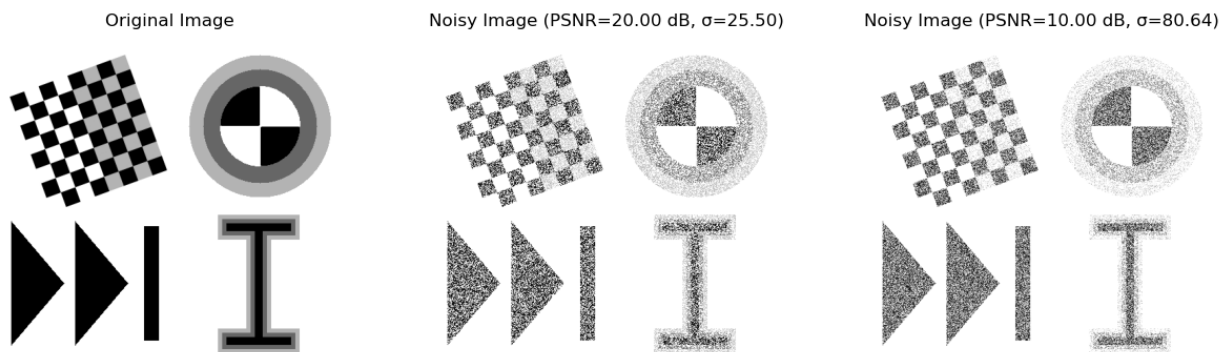
Εφαρμόζουμε τον θόρυβο στην εικόνα και παρουσιάζουμε τα αποτελέσματα

Listing 3: apply noise and show results

```

1  # Crate noisy images
2  image0_20, psnr_20, std_dev_20 = PSNRnoise(image0, 20)
3  image0_10, psnr_10, std_dev_10 = PSNRnoise(image0, 10)
4
5  # Display original and noisy images
6  plt.figure(figsize=(10, 5))
7
8  plt.subplot(1, 3, 1)
9  plt.imshow(image0_20, cmap='gray')
10 plt.title('Original Image')
11 plt.axis('off')
12
13 plt.subplot(1, 3, 2)
14 plt.imshow(image0_20, cmap='gray')
15 plt.title('Noisy Image (PSNR={:.2f} dB, {:.2f})'.format(psnr_20, std_dev_20))
16 plt.axis('off')
17
18 plt.subplot(1, 3, 3)
19 plt.imshow(image0_10, cmap='gray')
20 plt.title('Noisy Image (PSNR={:.2f} dB, {:.2f})'.format(psnr_10, std_dev_10))
21 plt.axis('off')
22
23 plt.subplots_adjust(right=1.3)
24 plt.show()
25

```



Σχήμα 1: Η αρχική εικόνα, εικόνα με γκαουσιανό θόρυβο PSNR=20dB, εικόνα με γκαουσιανό θόρυβο PSNR=10dB

1.2 Υλοποίηση Αλγορίθμων Ανίχνευσης Ακμών

1.2.1 Υλοποιήθηκαν οι δύο συναρτήσεις που επιστρέφουν τις ζητούμενες αποκρίσεις. Και οι δύο συναρτήσεις χρησιμοποιούν αυθαίρετα $\sigma = 5$. Το βέλτιστο σ θα βρεθεί αργότερα στην εργασία.

Listing 4: Implementation of G() (Gaussian) and LoG() (Laplacian of Gaussian) functions

```
1 SIGMA = 5 # Standard deviation of the Gaussian kernel
2 KSIZE = int(np.ceil(3*SIGMA))*2 + 1 # Kernel size
3
4 def G():
5     std_dev = SIGMA
6     ksize = KSIZE
7     kernel_1d = cv2.getGaussianKernel(ksize, std_dev)
8     kernel_2d = kernel_1d @ kernel_1d.T
9     return kernel_2d
10
11 def LoG():
12     ksize = KSIZE
13     sigma = SIGMA
14     x, y = np.meshgrid(np.arange(-ksize//2, ksize//2 + 1), np.arange(-ksize//2, ksize//2 + 1))
15     norm2 = x**2 + y**2
16     kernel = -(1.0 / (np.pi * sigma**4)) * (1 - norm2 / (2.0 * sigma**2)) * np.exp(-norm2 / (2.0 * sigma**2))
17
18     # Normalize Kernel
19     kernel -= kernel.mean()
20
21     return kernel
22
```

1.2.2 Εφαρμόζουμε τα ζητούμενα φίλτρα L_1 και L_2 και εμφανίζουμε τα αποτελέσματα. Στην προκειμένη περίπτωση χρησιμοποιείται η θορυβοποιημένη εικόνα με PSNR=20dB.

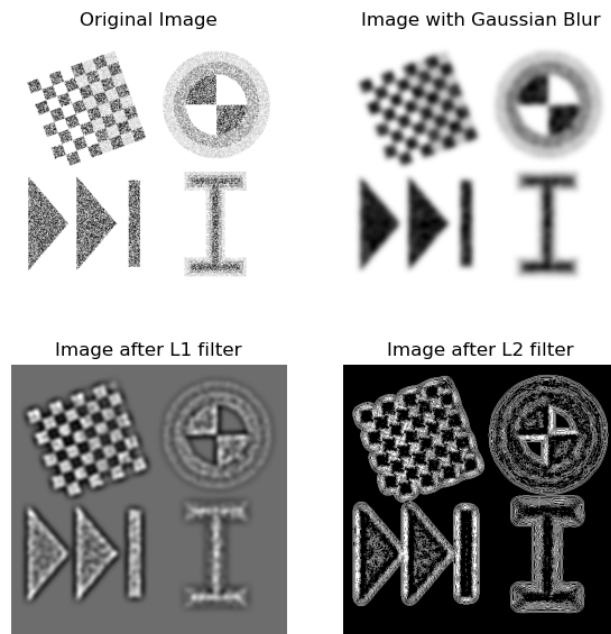
Listing 5: apply L1 and L2 filters and show results

```
1 OG_IMAGE = image0_20 # You can change this to image_0 image0_20 or image0_10
2
3 # LoG filter
4 image0_L1 = cv2.filter2D(OG_IMAGE, cv2.CV_64F, LoG())
5
6 # Morphological filters
7 image0_blur = cv2.filter2D(OG_IMAGE, -1, G())
8 B = np.array([[0, 1, 0],
9               [1, 1, 1],
10              [0, 1, 0]], dtype=np.uint8)
11 image0_L2 = cv2.dilate(image0_blur, B) + cv2.erode(image0_blur, B) - 2*image0_blur
12
13 # Show results
14 plt.figure(figsize=(7, 7))
15
16 plt.subplot(2, 2, 1)
17 plt.title("Original Image")
18 plt.imshow(OG_IMAGE, cmap='gray')
19 plt.axis('off')
20
21 plt.subplot(2, 2, 2)
22 plt.title("Image with Gaussian Blur")
23 plt.imshow(image0_blur, cmap='gray')
24 plt.axis('off')
25
26 plt.subplot(2, 2, 3)
27 plt.title("Image after L1 filter")
28 plt.imshow(image0_L1, cmap='gray')
29 plt.axis('off')
30
31 plt.subplot(2, 2, 4)
```

```

32 plt.title("Image after L2 filter")
33 plt.imshow(image0_L2, cmap='gray')
34 plt.axis('off')
35
36 plt.show()
37

```



Σχήμα 2: Η αρχική εικόνα, μετά από το φίλτρο G, μετά το φίλτρο LoG, μετά τα μορφολογικά φίλτρα

1.2.3 Γίνεται η προσέγγιση των zero-crossings. Χρησιμοποιείται ο προτεινόμενος αλγόριθμος, δημιουργείται δηλαδή η δυαδική εικόνα και εντοπίζεται το περίγραμμα με χρήση των κατάλληλων μορφολογικών τελεστών. Εμφανίζονται τα αποτελέσματα

```

1  # Binary sign image
2  image0_X1 = (image0_L1 >= 0).astype(np.uint8)
3
4  # Find border
5  image0_Y1 = cv2.dilate(image0_X1, B) - cv2.erode(image0_X1, B)
6
7  # Show results
8  plt.figure(figsize=(10, 7))
9
10 plt.subplot(1, 3, 1)
11 plt.title("L1")
12 plt.imshow(image0_L1, cmap='gray')
13 plt.axis('off')
14
15 plt.subplot(1, 3, 2)
16 plt.title("X1")
17 plt.imshow(image0_X1, cmap='gray')
18 plt.axis('off')
19
20 plt.subplot(1, 3, 3)
21 plt.title("Y1")
22 plt.imshow(image0_Y1, cmap='gray')
23 plt.axis('off')
24
25 plt.show()
26

```



Σχήμα 3:
L1: Η εικόνα μετά το φίλτρο LoG
X1: Η δυαδική εικόμα της L1
Y1: Το περίγραμμα της X1

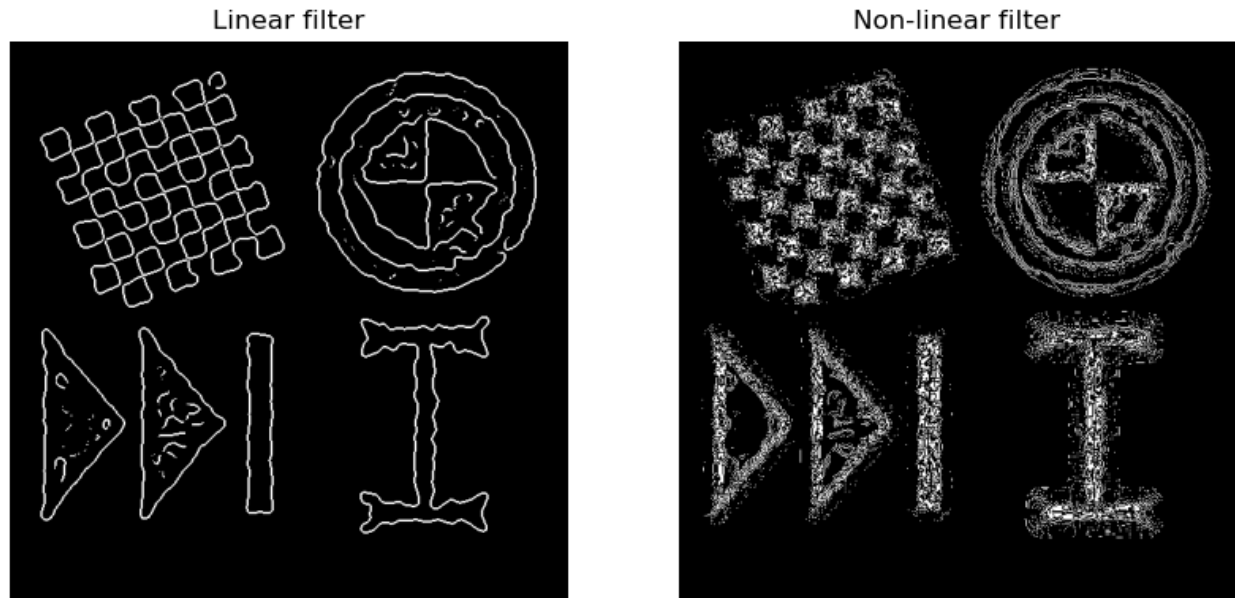
1.2.4 Απορρίπτονται οι ακμές σε ομαλές περιοχές. Θεωρούμε αυθαίρετα $\theta_{edge} = 0.1$. Το βέλτιστο θ_{edge} θα βρεθεί αργότερα στην εργασία. Εμφανίζονται τα αποτελέσματα.

Listing 6: keep only edges in 'steep' areas. Show results

```

1 THETA_EDGE = 0.1
2
3 gradient = np.gradient(image0_blur)
4 gradient_norm = np.sqrt(gradient[0]**2 + gradient[1]**2)
5 image0_edges1 = (image0_Y1==1) & (gradient_norm>THETA_EDGE*np.max(gradient_norm))
6 image0_edges2 = (image0_L2==1) & (gradient_norm>THETA_EDGE*np.max(gradient_norm))
7
8 plt.figure(figsize=(10,5))
9
10 plt.subplot(1,2,1)
11 plt.title("Linear filter")
12 plt.imshow(image0_edges1, cmap='gray')
13 plt.axis('off')
14
15 plt.subplot(1,2,2)
16 plt.title("Non-linear filter")
17 plt.imshow(image0_edges2, cmap='gray')
18 plt.axis('off')
19
20 plt.show()
21

```



Σχήμα 4:

Αριστερά: Το τελικό περίγραμμα της εικόνας με χρήση γραμμικού φίλτρου

Δεξιά: Το τελικό περίγραμμα της εικόνας με χρήση μη γραμμικού μορφολογικού φίλτρου

Τέλος, συνθέτουμε τα παραπάνω βήματα στη ζητούμενη συνάρτηση EdgeDetect()

Listing 7: Implementation of EdgeDetect() function

```

1  def EdgeDetect(image, sigma, theta_edge, linear=True):
2      ksize = int(np.ceil(3*sigma))*2 + 1
3
4      # define Gaussian kernel
5      kernel_1d = cv2.getGaussianKernel(ksize, sigma)
6      G = kernel_1d @ kernel_1d.T
7
8      # create Gaussian blurred image
9      image_blur = cv2.filter2D(image, -1, G)
10
11     # define morphological filters kernel
12     B = np.array([[0, 1, 0],
13                   [1, 1, 1],
14                   [0, 1, 0]], dtype=np.uint8)
15
16     if linear:
17         # define LoG kernel
18         x, y = np.meshgrid(np.arange(-ksize//2, ksize//2 + 1), np.arange(-ksize//2, ksize//2 +
19     1))
20         norm2 = x**2 + y**2
21         LoG = -((1.0 / (np.pi * sigma**4)) * (1 - norm2 / (2.0 * sigma**2)) * np.exp(-norm2 /
22     (2.0 * sigma**2)))
23         LoG -= LoG.mean()
24
25     # convolve image with LoG
26     image_L1 = cv2.filter2D(image, cv2.CV_64F, LoG)
27
28     # Binary sign image
29     image_X1 = (image_L1 >= 0).astype(np.uint8)
30
31     # Find border
32     image_Y1 = cv2.dilate(image_X1, B) - cv2.erode(image_X1, B)
33
34     D = image_Y1

```

```

33
34     else:
35         # apply morphological filters
36         image_L2 = cv2.dilate(image_blur, B) + cv2.erode(image_blur, B) - 2*image_blur
37
38         D = image_L2
39
40         # define norm of gradient of blurred image
41         gradient = np.gradient(image_blur)
42         gradient_norm = np.sqrt(gradient[0]**2 + gradient[1]**2)
43
44         # keep only steep edges
45         D = (D == 1) & (gradient_norm > theta_edge*np.max(gradient_norm))
46
47     return D
48

```

1.3 Αξιολόγηση Αποτελεσμάτων Ανίχνευσης Ακμών

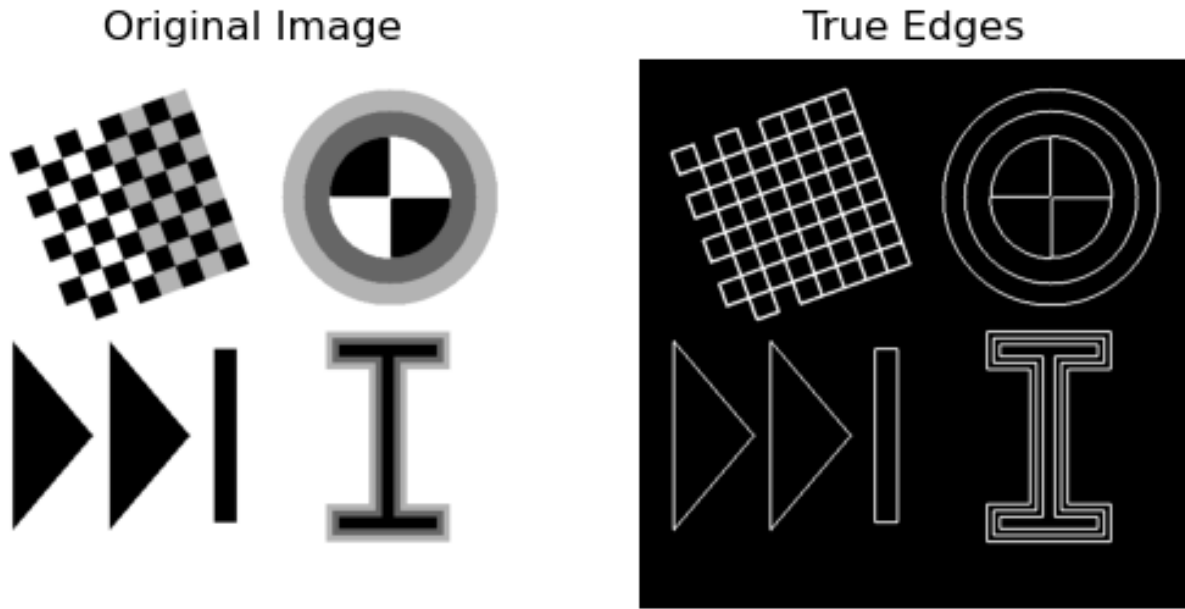
1.3.1 Εφαρμόζουμε στην εικόνα τους κατάλληλους μορφολογικούς τελεστές και καταφλιποιούμε το αποτέλεσμα, κατασκευάζουμε την εικόνα T, με τις πραγματικές ακμές. Θεωρώντας $\theta_{ealedge1}$, παίρνουμε το προσδοκώμενο αποτέλεσμα.

Listing 8: locate real edges

```

1
2     THETA_REALEDGE = 1
3
4     # perform edge operator
5     M = cv2.dilate(image0, B) - cv2.erode(image0, B)
6
7     # binary image
8     T = M > THETA_REALEDGE
9
10    # show result
11    plt.subplot(1,2,1)
12    plt.title("Original Image")
13    plt.imshow(image0, cmap='gray')
14    plt.axis('off')
15
16    plt.subplot(1,2,2)
17    plt.title("True Edges")
18    plt.imshow(T, cmap='gray')
19    plt.axis('off')
20
21    plt.show()
22

```



Σχήμα 5: Η αρχική εικόνα και οι ακμές της

1.3.2 Ορίζεται η συνάρτηση `score()`, που δέχεται μια εικόνα με τις εκτιμώμενες ακμές και την εικόνα με τις πραγματικές ακμές και υπολογίζει την παράμετρο C , σύμφωνα με τον δωσμένο τύπο.

Listing 9: implementation of `score()` function

```

1  def score(detected_edges, true_edges):
2      D = detected_edges
3      T = true_edges
4      precision = ((D==1) & (T==1)).sum()/D.sum()
5      recall = ((D==1) & (T==1)).sum()/T.sum()
6      return (precision + recall)/2
7

```

1.3.3 Εφαρμόζουμε την συνάρτηση `EdgeDetect()` στην εικόνα με PSNR=20dB και PSNR=10dB αντίστοιχα, δοκιμάζοντας χειροκίνητα διαφορετικές παραμέτρους, με μέθοδο όμοια του αλγορίθμου Gradient Descent. Οι τιμές που οδήγησαν στα (τοπικά) βέλτιστα `score` φαίνονται στην εικόνα παρακάτω.

Listing 10: tweak parameters

```

1  # PSNR==20
2  linear_20 = EdgeDetect(image0_20, 1.5, 0.2, True)
3  non_linear_20 = EdgeDetect(image0_20, 1.5, 0.2, False)
4
5  # PSNR==10
6  linear_10 = EdgeDetect(image0_10, 3, 0.2, True)
7  non_linear_10 = EdgeDetect(image0_10, 3, 0.2, False)
8
9  # Show results
10 plt.figure(figsize=(10,10))
11
12 plt.subplot(2,2,1)
13 plt.suptitle("Detected Edges")
14 plt.title(f"PSNR=20dB, =1.5, =0.2, Linear (C={score(linear_20, T):.4f})")
15 plt.imshow(linear_20, cmap='gray')
16 plt.axis('off')
17
18 plt.subplot(2,2,2)

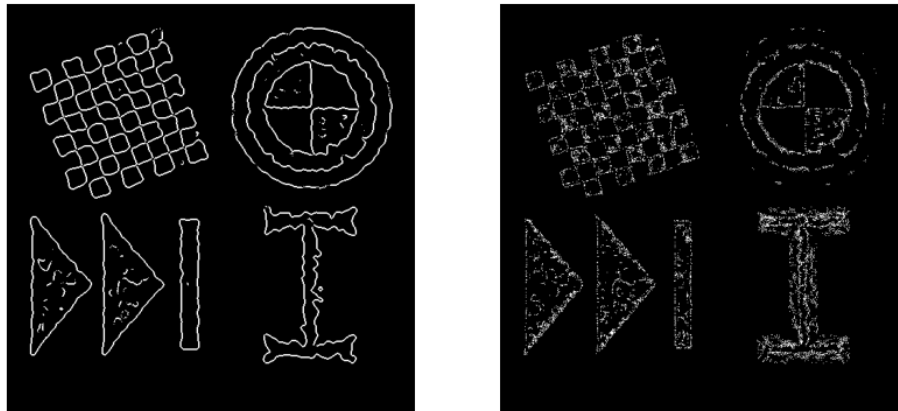
```

```

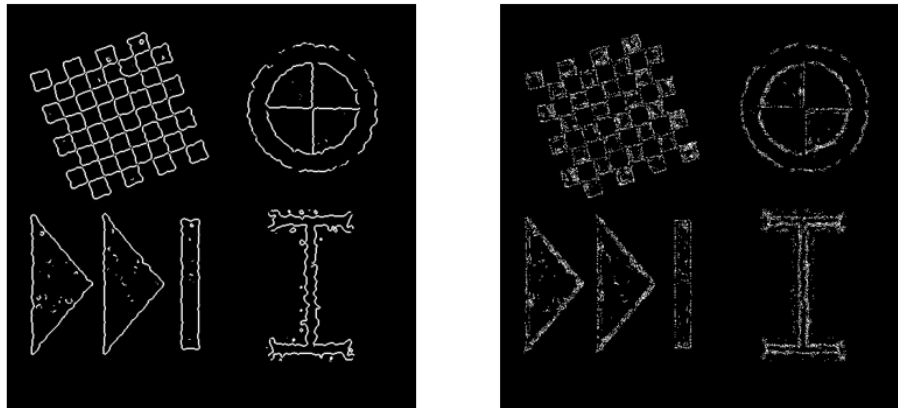
19 plt.title(f"PSRN=20dB, =1.5, =0.2, Non-Linear (C={score(non_linear_20, T):.4f})")
20 plt.imshow(non_linear_20, cmap='gray')
21 plt.axis('off')
22
23 plt.subplot(2,2,3)
24 plt.title(f"PSRN=10dB, =3, =0.2, Linear (C={score(linear_10, T):.4f})")
25 plt.imshow(linear_10, cmap='gray')
26 plt.axis('off')
27
28 plt.subplot(2,2,4)
29 plt.title(f"PSRN=10dB, =3, =0.2, Non-Linear (C={score(non_linear_10, T):.4f})")
30 plt.imshow(non_linear_10, cmap='gray')
31 plt.axis('off')
32
33 plt.show
34

```

PSRN=20dB, $\sigma=4.5$, $\theta=0.16$, Linear (C=0.5435) PSRN=20dB, $\sigma=3$, $\theta=0.2$, Non-Linear (C=0.2663)



PSRN=10dB, $\sigma=3.2$, $\theta=0.2$, Linear (C=0.6145) PSRN=10dB, $\sigma=3.1$, $\theta=0.18$, Non-Linear (C=0.2727)



Σχήμα 6: Τα τελικά αποτελέσματα των δύο αλγορίθμων (L1&L2), μετά από δοκιμές πολλών τιμών παραμέτρων

Παρατηρήσεις:

- Οι "βέλτιστες" αυτές τιμές βρέθηκαν προσεγγιστικά χρησιμοποιώντας χειροκίνητη μέθοδο. Για να βρεθούν οι πραγματικά βέλτιστες παράμετροι θα πρέπει να εφαρμοστεί επαναληπτικά ο αλγόριθμος Gradient Descent, από διαφορετικά σημεία εκκίνησης κάθε φορά, για να αποφευχθούν τυχόν τοπικά βέλτιστες λύσεις.
- Ο αλγόριθμος που αξιοποιεί το γραμμικό φίλτρο LoG έχει οδηγή σε σημαντικά καλύτερα αποτελέσματα (τουλάχιστον με το εύρος παραμέτρων που δοκιμάστηκαν)

- Το γραμμικό φίλτρο LoG φαίνεται να προσεγγίζει τις ακμές με συνεχείς γραμμές. Αντιθέτως, το μορφολογικό φίλτρο φαίνεται να κατασκευάζει τις ακμές με ανεξάρτητα μεταξύ τους σημεία.
- Επαναλαμβάνοντας το πρόγραμμα με ακριβώς τις ίδιες παραμέτρους οδηγούμαστε κάθε φορά σε ελαφρώς διαφορετικά αποτελέσματα. Θεωρώ ότι αυτό οφείλεται στο στάδιο της θορυβοποίησης που περιλαμβάνει, έστω σε μικρή κλίμακα, τον παράγοντα της "τύχης".

1.4 Εφαρμογή των Αλγορίθμων Ανίχνευσης Ακμών σε Πραγματικές Εικόνες

1.4.1 Φορτώνουμε την εικόνα 'butterfly.jpg' και εφαρμόζουμε τη συνάρτηση EdgeDetect().

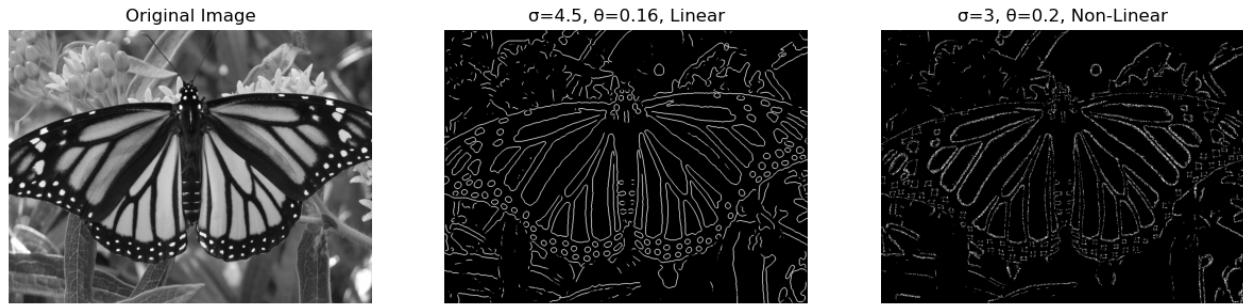
Listing 11: load butterfly.jpg and apply EdgeDetect()

```

1  # Load butterfly image
2  image1 = cv2.imread('butterfly.jpg')
3
4  # Convert the image from BGR to GRAY
5  image1= cv2.cvtColor(image1, cv2.COLOR_BGR2GRAY)
6
7  # tweak parameters
8  linear_sigma = 4.5
9  linear_theta = 0.16
10
11 non_linear_sigma = 3
12 non_linear_theta = 0.2
13
14 # Perform Edge Detection
15 linear = EdgeDetect(image1, linear_sigma, linear_theta, True)
16 non_linear = EdgeDetect(image1, non_linear_sigma, non_linear_theta, False)
17
18 # Show Results
19 plt.figure(figsize=(15,5))
20
21 plt.subplot(1,3,1)
22 plt.title("Original Image")
23 plt.imshow(image1, cmap='gray')
24 plt.axis('off')
25
26 plt.subplot(1,3,2)
27 plt.title(f"={linear_sigma}, ={linear_theta}, Linear")
28 plt.imshow(linear, cmap='gray')
29 plt.axis('off')
30
31 plt.subplot(1,3,3)
32 plt.title(f"={non_linear_sigma}, ={non_linear_theta}, Non-Linear")
33 plt.imshow(non_linear, cmap='gray')
34 plt.axis('off')
35
36 plt.show()
37

```

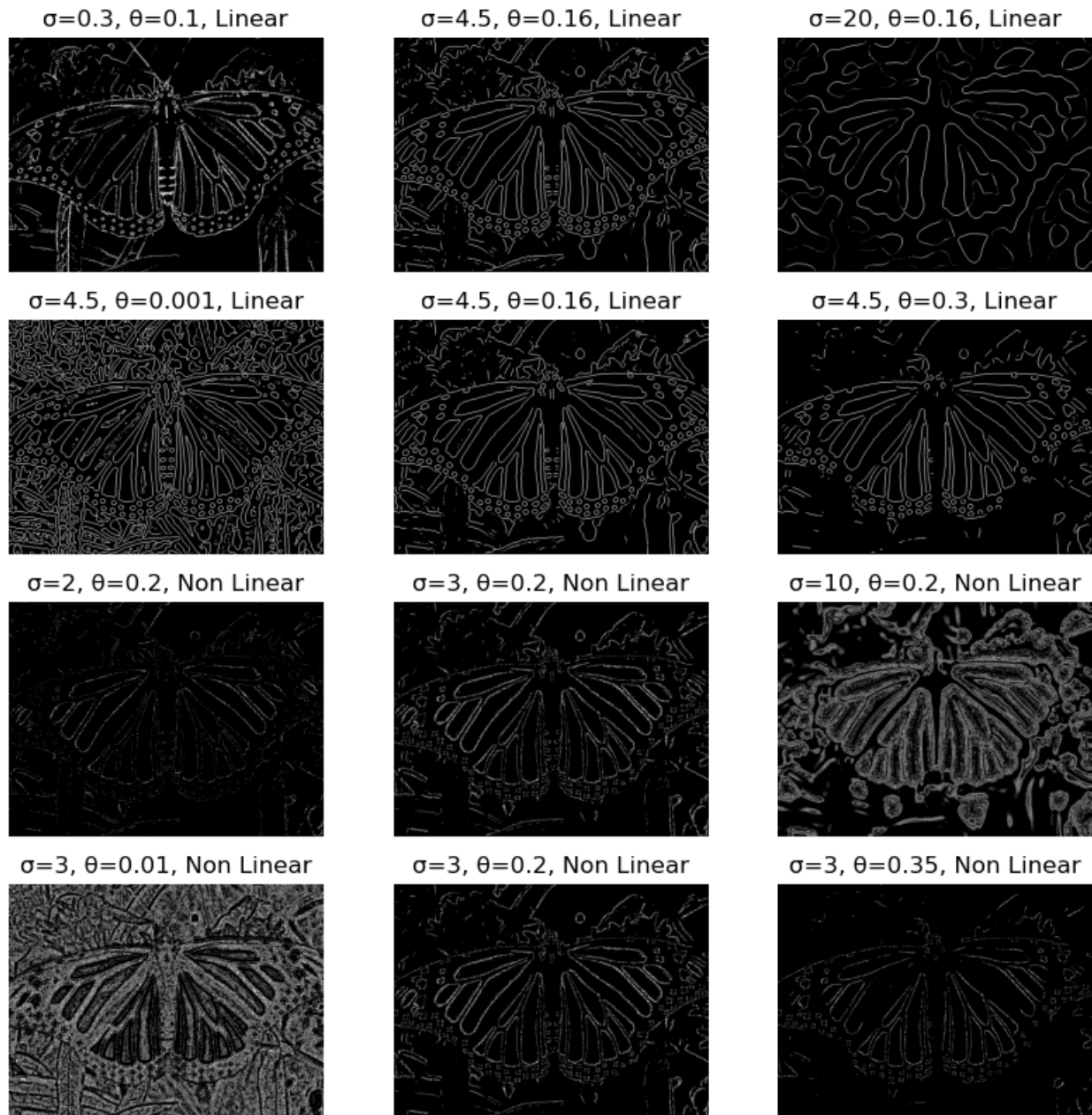
1.4.2 Δοκιμάζουμε όπως προηγουμένως διαφορετικές τιμές για τις παραμέτρους του αλγορίθμου. Οι βέλτιστες που βρεθήκαν φαίνονται στην εικόνα



Σχήμα 7: Τα τελικά αποτελέσματα των δύο αλγορίθμων (L1&L2), μετά από δοκιμές πολλών τιμών παραμέτρων

Παρατηρήσεις:

- Όταν έχουμε μικρό σ_n , τόσο περισσότερες λεπτομέρειες περιλαμβάνονται στις ακμές. στην περίπτωση της πραγματικής εικόνας, το αποτέλεσμα είναι αρκετά ικανοποιητικό,. Αν προσθέσουμε όμως θόρυβο, τότε αυτός λαμβάνεται ως ακμή, πράγμα ανεπιθύμητο.
- Όσο αυξάνουμε το σ_n , συμπεριλαμβάνονται όλο και λιγότερες λεπτομέρειες και οι ακμές που βρίσκουμε ομαλοποιούνται.
- Όμοια με το σ_n , έτσι και το θ_{edge} όταν είναι πολύ μικρό συμπεριλαμβάνονται πολλές "λεπτομέρειες". Εδώ όμως οι "λεπτομέρειες" είναι απλά περιοχές με ελαφρώς απότομη μεταβολή στην φωτεινότητα και που δε θα θεωρούσαμε ακμές. Το τελικό αποτέλεσμα είναι περιοχές με ένα "θορυβώδες" πλήθος ακμών. Σε κάθε περίπτωση λοιπόν το υπερβολικά μικρό θ_{edge} είναι ανεπιθύμητο.
- Καθώς μεγαλώνουμε το θ_{edge} , ο αριθμός αυτών των περιοχών με ασήμαντη μεταβολή στη φωτεινότητα μειώνεται, φτάνοντας ένα βέλτιστο σημείο. Αν συνεχίσουμε να αυξάνουμε περαιτέρω, θα αρχίσουν να περιορίζονται ακόμα και οι επιθυμητές ακμές, μέχρι να φτάσουμε σε μια τελείως σκοτεινή εικόνα.



Σχήμα 8: Η εικόνα butterfly.jpg, μετά από δοκιμές διαφόρων παραμέτρων

Μέρος 2: Ανίχνευση Σημείων Ενδιαφέροντος

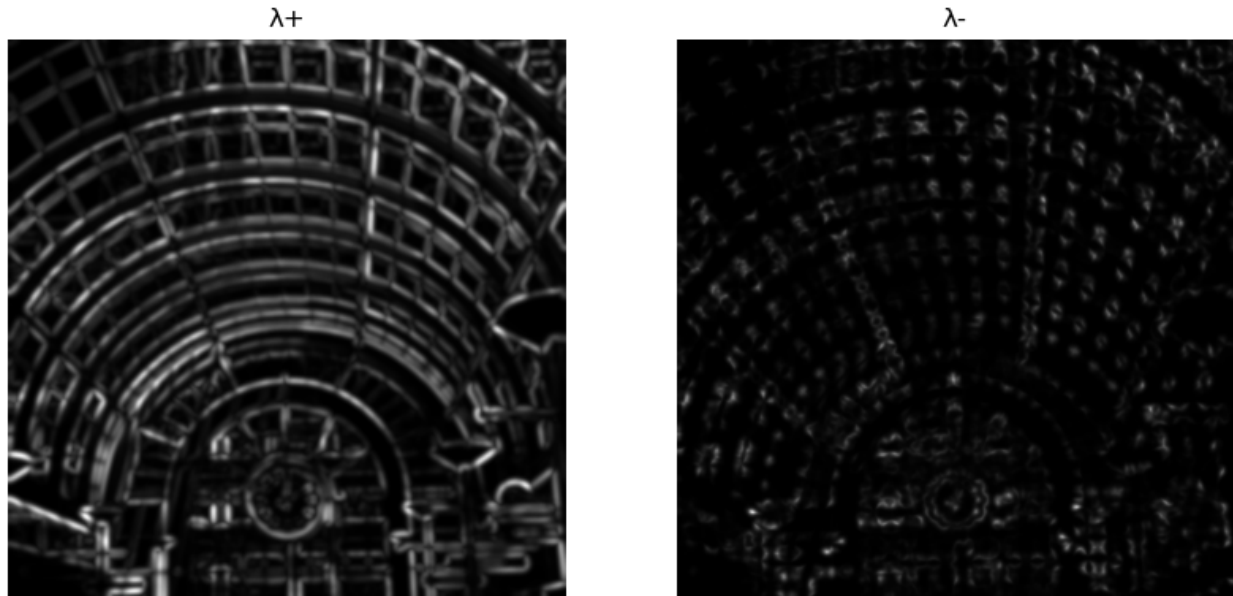
2.1 Ανίχνευση γωνιών

2.1.1 Υπολογίζονται τα στοιχεία του δομικού τελεστή J .

```
1 # Create structure tensor
2 I_sigma = cv2.GaussianBlur(image1, (0,0), SIGMA)
3 I_sy, I_sx = np.gradient(I_sigma)
4 J1 = cv2.GaussianBlur(I_sx*I_sx, (0,0), R0)
5 J2 = cv2.GaussianBlur(I_sx*I_sy, (0,0), R0)
6 J3 = cv2.GaussianBlur(I_sy*I_sy, (0,0), R0)
7
8 # Calculate eigenvalues
9 lamda_plus = (J1 + J3 + np.sqrt(np.square(J1-J3) + 4*np.square(J2)))/2
10 lamda_minus = (J1 + J3 - np.sqrt(np.square(J1-J3) + 4*np.square(J2)))/2
11
```

2.1.2 Υπολογίζουμε τις ιδιοτιμές λ_+ , λ_- , βγάζοντας αναμενόμενα αποτελέσματα

```
1 # Create structure tensor
2 I_sigma = cv2.GaussianBlur(image1, (0,0), SIGMA)
3 I_sy, I_sx = np.gradient(I_sigma)
4 J1 = cv2.GaussianBlur(I_sx*I_sx, (0,0), R0)
5 J2 = cv2.GaussianBlur(I_sx*I_sy, (0,0), R0)
6 J3 = cv2.GaussianBlur(I_sy*I_sy, (0,0), R0)
7
8 # Calculate eigenvalues
9 lamda_plus = (J1 + J3 + np.sqrt(np.square(J1-J3) + 4*np.square(J2)))/2
10 lamda_minus = (J1 + J3 - np.sqrt(np.square(J1-J3) + 4*np.square(J2)))/2
11
12 # Show image
13 plt.figure(figsize=(10,10))
14
15 plt.subplot(1,2,1)
16 plt.imshow(lamda_plus, cmap='gray')
17 plt.title("+")
18 plt.axis('off')
19 plt.subplot(1,2,2)
20 plt.imshow(lamda_minus, cmap='gray')
21 plt.title("-")
22 plt.axis('off')
23
24 plt.show()
25
26 # Some cleanup
27 del I_sigma, I_sy, I_sx, J1, J2, J3
28
```



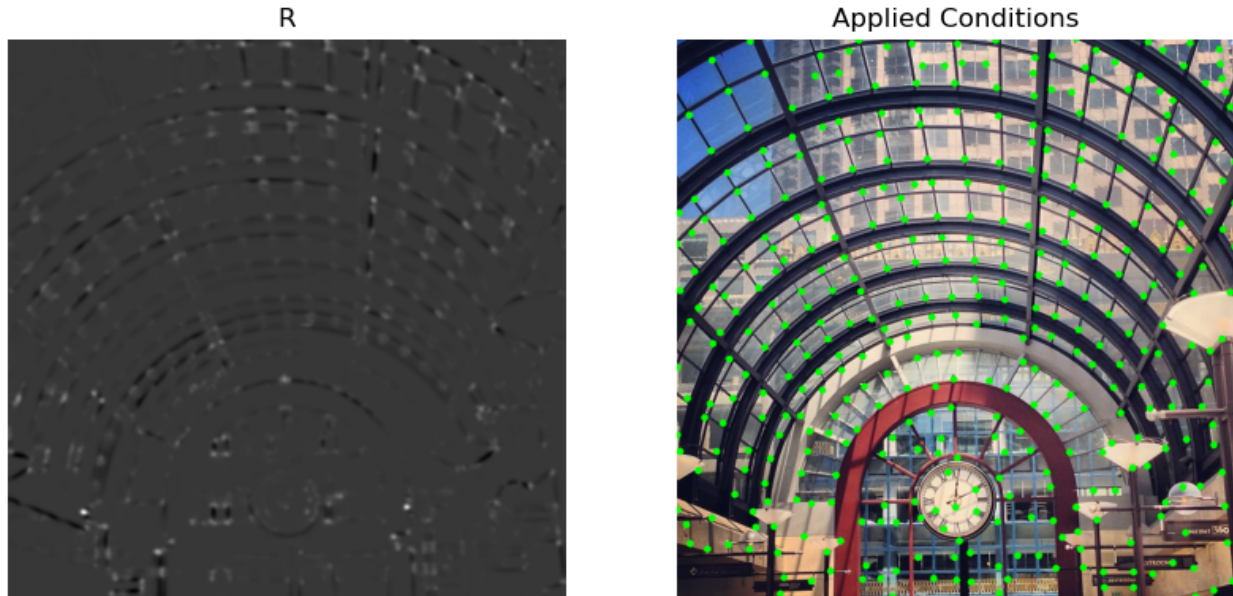
Σχήμα 9: $\lambda+$ και $\lambda-$

2.1.3 Υπολογίζουμε το κριτήριο γωνιότητας και εφαρμόζουμε τις δύο συνθήκες. Τέλος, εμφανίζουμε την εικόνα με πράσινες τελείες στα σημεία που απέμειναν.

```

1  # Coreerness criterion
2  R = lamda_plus*lamda_minus - K*np.square(lamda_plus + lamda_minus)
3
4  # Condition 1
5  ns = np.ceil(3*SIGMA)*2+1
6  B_sq = utils.disk_strel(ns)
7  Cond1 = ( R==cv2.dilate(R,B_sq) )
8
9  # Condition 2
10 Cond2 = ( R > THETA_CORN*np.max(R))
11
12 corners = np.logical_and(Cond1, Cond2)
13
14 # Put green dots on corners
15 marked_image = image1_rbg.copy()
16 for y, x in zip(*np.where(corners)):
17     cv2.circle(marked_image, (x, y), 4, (0, 255, 0), -1)
18
19 # Show image
20 plt.figure(figsize=(10,10))
21 plt.subplot(1,2,1)
22 plt.imshow(R, cmap='gray')
23 plt.title("R")
24 plt.axis('off')
25 plt.subplot(1,2,2)
26 plt.imshow(marked_image, cmap='gray')
27 plt.title("Applied Conditions")
28 plt.axis('off')
29 plt.show()
30

```



Σχήμα 10: Μονοκλιμακωτή εύρεση γωνιών

2.2 Πολυκλιμακωτή Ανίχνευση γωνιών

2.2.1 Αφού υλοποιήσουμε μια συνάρτηση που περιλαμβάνει τον αλγόριθμο του μέρους 2.1, τρέχουμε τον αλγόριθμο για πολλές κλίμακες.

```

1  # Run CornerDetect for multiple scales
2  sigmas = [S**i * SIGMA for i in range(N)]
3  ros = [S**i * R0 for i in range(N)]
4  image_corners = [CornerDetect(image1, sigma, ro, THETA_CORN) for sigma, ro in zip(sigmas, ros)]
5  ]

```

2.2.2 Υπολογίζουμε την κανονικοποιημένη LoG για καθεμία από τις κλίμακες. Για κάθε σημείο (x,y) που έχουμε βρει, ελέγχουμε αν $LoG(x, y, \sigma_i) > LoG(x, y, \sigma_{i+1})$ και $LoG(x, y, \sigma_i) > LoG(x, y, \sigma_{i-1})$. Αν δεν ισχύει η συνθήκη, το απορρίπτουμε. Τέλος, οπτικοποιούμε το αποτέλεσμα χρησιμοποιώντας τη συνάρτηση `interest_points_visualization()`.

```

1  # Create Normalized LoG
2  LoG = []
3  for i in range(N):
4      I_sigma = cv2.GaussianBlur(image1, (0,0), sigmas[i])
5      I_sy, I_sx = np.gradient(I_sigma)
6      Lxx = np.gradient(I_sx)[1]
7      Lyy = np.gradient(I_sy)[0]
8      LoG.append((sigmas[i]**2)*abs(Lxx+Lyy))
9
10 # For each scale, keep only corners that are maximum in area of 2 scales
11 for i in range(N):
12     for y, x in zip(*np.where(image_corners[i])):
13         bool1 = (LoG[i][y, x] > LoG[i-1][y, x]) if i>0 else True #if-else ensures that index stays
14         bool2 = (LoG[i][y, x] > LoG[i+1][y, x]) if i<len(LoG)-1 else True
15         image_corners[i][y, x] = bool1 and bool2
16
17 # Show result
18 kp_data = []
19 for i in range(N):
20     for y, x in zip(*np.where(image_corners[i])):
21         kp_data.append([x, y, sigmas[i]])

```

```
utils.interest_points_visualization(image1_rgb, kp_data)
```



Σχήμα 11: Πολυκλιμακωτό corner detection

2.3 Ανίχνευση Blobs

2.3.1 Υπολογίζονται οι μερικές παράγωγοι και το κριτήριο R

2.3.2 Χρησιμοποιώντας τις ίδιες συνθήκες με το μέρος 2.1, καταλήγουμε σε ορισμένα blobs, τα οποία σημαδεύουμε με πράσινες τελείες.

```
1 I_sigma = cv2.GaussianBlur(image2, (0,0), SIGMA)
2 I_sy, I_sx = np.gradient(I_sigma)
3
4 Lxx = np.gradient(I_sx)[1]
5 Lyy = np.gradient(I_sy)[0]
6 Lxy = np.gradient(I_sx)[0]
7
8 R = Lxx*Lyy - Lxy**2
9
10 # Condition 1
11 ns = np.ceil(3*SIGMA)*2+1
12 B_sq = utils.disk_strel(ns)
13 Cond1 = ( R==cv2.dilate(R,B_sq) )
14
15 # Condition 2
16 Cond2 = ( R > THETA_BLOB*np.max(R) )
17
18 blobs = np.logical_and(Cond1, Cond2)
19
20 # Put green dots on blobs
21 marked_image2 = image2_rgb.copy()
22 for y, x in zip(*np.where(blobs)):
23     cv2.circle(marked_image2, (x, y), 4, (0, 255, 0), -1)
24
```



Σχήμα 12: Μονοκλιμακωτό blob detection

2.4 Πολυκλιμακωτή Ανίχνευση Blobs

2.4.1 Όπως και προηγουμένως, υλοποιούμε τη συνάρτηση *BlobDetect*(). Ακολουθώντας όμοια διαδικασία, υλοποιούμε το normalized *LoG* και κρατάμε μόνο τα σημεία που είναι μέγιστα σε σχέση με τις γειτονικές τους κλίμακες. Οπτικοποιούμε το αποτέλεσμα.

```

1  # Run BlobDetect for multiple scales
2  sigmas = [S**i * SIGMA for i in range(N)]
3  ros = [S**i * R0 for i in range(N)]
4  image_blobs = [BlobDetect(image2, sigma, THETA_BLOB) for sigma, ro in zip(sigmas, ros)]
5
6  # Create Normalized LoG
7  LoG = []
8  for i in range(N):
9      I_sigma = cv2.GaussianBlur(image2, (0,0), sigmas[i])
10     I_sy, I_sx = np.gradient(I_sigma)
11     Lxx = np.gradient(I_sx)[1]
12     Lyy = np.gradient(I_sy)[0]
13     LoG.append((sigmas[i]**2)*abs(Lxx+Lyy))
14
15 # For each scale, keep only corners that are maximum in area of 2 scales
16 for i in range(N):
17     for y, x in zip(*np.where(image_blobs[i])):
18         bool1 = (LoG[i][y, x] > LoG[i-1][y, x]) if i>0 else True #if-else ensures that index stays
19         within range
20         bool2 = (LoG[i][y, x] > LoG[i+1][y, x]) if i<len(LoG)-1 else True
21         image_blobs[i][y, x] = bool1 and bool2
22
23 # Show result
24 kp_data = []
25 for i in range(N):
26     for y, x in zip(*np.where(image_blobs[i])):
27         kp_data.append([x, y, sigmas[i]])
28
29     utils.interest_points_visualization(image2_rgb, kp_data)

```



Σχήμα 13: Πολυκλιμακωτό blob detection

2.5 Επιτάχυνση με την χρήση Box Filters και Ολοκληρωτικών Εικόνων (Integral Images)

2.5.1 Χρησιμοποιώντας τη συνάρτηση `np.cumsum()` κατά τους άξονες x και y της αρχικής εικόνας (σε grayscale), υπολογίζουμε την αντίστοιχη πολυκλιμακωτή εικόνα.

```

1  # Define parameters
2  SIGMA_BOX_FILTERS = 2
3
4  # Calculate Integral image
5  image2_integral = np.cumsum(np.cumsum(image2, axis=0), axis=1)
6  image2_integral = image2_integral.astype(np.float64)
7
8  plt.imshow(image2_integral, cmap='gray')
9  plt.axis('off')
10

```



Σχήμα 14: Η ολοκληρωτική εικόνα της εικόνας εισόδου

2.5.2 Υλοποιούμε τη βοηθητική συνάρτηση `int_sum()`, που δέχεται σαν ορίσματα τις διαστάσεις ενός παραθύρου, μια ολοκληρωτική εικόνα και αρχικές συντεταγμένες (x,y) και υπολογίζει το άθροισμα των σημείων της αρχικής εικόνας στο δοσμένο παράθυρο, χρησιμοποιώντας τον τύπο (18) της εκφώνησης. Ύστερα, προκειμένου να υπολογίσουμε τα D_{xx} , D_{yy} , και D_{xy} , διατρέχουμε κάθε σημείο της εικόνας και καλούμε τη συνάρτηση `int_sum()` (με αρχικό το διατρεχόμενο σημείο), υπολογίζοντας τα αθροίσματα των κατάλληλων περιοχών, ανάλογα με τη συνάρτηση που προσεγγίζουμε. Για το κάθε σημείο, προσθέτουμε ή αφαιρούμε τις περιοχές κατάλληλα, σύμφωνα με τους πυρήνες του σχήματος 2 της εκφώνησης.

```

1  # Integral image sum function
2  def int_sum(height, width, integral_image, y, x):
3  return integral_image[y, x] + integral_image[y+height, x+width] - integral_image[y+height, x]
   - integral_image[y, x+width]

```

```

4
5 # Apply box filters
6 n = 2*np.ceil(3*SIGMA_BOX_FILTERS)+1
7
8 dxx_height = int(4*np.floor(n/6)+1)
9 dxx_width = int(2*np.floor(n/6)+1)
10 dyy_height = int(2*np.floor(n/6)+1)
11 dyy_width = int(4*np.floor(n/6)+1)
12 dxy_width = int(2*np.floor(n/6)+1)
13 dxy_height = int(2*np.floor(n/6)+1)
14
15 image2_integral_pad = np.pad(image2_integral, ((dxx_height*2,0),(dyy_width*2,0)))
16 image_height, image_width = np.shape(image2)
17
18 # Dxx
19 Dxx = np.zeros(np.shape(image2))
20 for i in range(image_height):
21     for j in range(image_width):
22         Dxx[i,j] = int_sum(dxx_height, dxx_width, image2_integral_pad, i, j) \
23             - 2*int_sum(dxx_height, dxx_width, image2_integral_pad, i, j+dxx_width) \
24             + int_sum(dxx_height, dxx_width, image2_integral_pad, i, j+dxx_width*2)
25
26 # Dyy
27 Dyy = np.zeros(np.shape(image2))
28 for i in range(image_height):
29     for j in range(image_width):
30         Dyy[i,j] = int_sum(dyy_height, dyy_width, image2_integral_pad, i, j) \
31             - 2*int_sum(dyy_height, dyy_width, image2_integral_pad, i+dyy_height, j) \
32             + int_sum(dyy_height, dyy_width, image2_integral_pad, i+dyy_height*2, j)
33
34 # Dxy
35 Dxy = np.zeros(np.shape(image2))
36 for i in range(image_height):
37     for j in range(image_width):
38         Dxy[i,j] = int_sum(dxy_height, dxy_width, image2_integral_pad, i, j) \
39             - int_sum(dxy_height, dxy_width, image2_integral_pad, i+dxy_height, j) \
40             - int_sum(dxy_height, dxy_width, image2_integral_pad, i, j+dxy_width) \
41             + int_sum(dxy_height, dxy_width, image2_integral_pad, i+dxy_height, j+dxy_width)
42

```

2.5.3 Υπολογίζουμε το αντίστοιχο κριτήριο blob.

```

1 R = Dxx*Dyy - np.square(0.9*Dxy)
2

```

2.5.4 Για άλλη μια φορά, συνδιάζουμε τα προηγούμενα βήματα σε μια συνάρτηση (*FastBlobDetect()*), η οποία είναι ίδια με την *BlobDetect()*, μόνο που αντί για L_{xy} χρησιμοποιεί τις προσεγγίσεις D_{xy} , καθώς και χρησιμοποιεί το αντίστοιχο προσεγγιστικό R . Χρησιμοποιούμε τη συνάρτηση και εφαρμόζουμε ακριβώς την ίδια διαδικασία με το μέρος 2.4.1.

```

1 # Run FastBlobDetect for multiple scales
2 sigmas = [S**i * SIGMA for i in range(N)]
3 ros = [S**i * R0 for i in range(N)]
4 image_blobs = [FastBlobDetect(image2, sigma, THETA_BLOB) for sigma, ro in zip(sigmas, ros)]
5
6 # Create Normalized LoG
7 LoG = []
8 for i in range(N):
9     I_sigma = cv2.GaussianBlur(image2, (0,0), sigmas[i])
10    I_sy, I_sx = np.gradient(I_sigma)
11    Lxx = np.gradient(I_sx)[1]
12    Lyy = np.gradient(I_sy)[0]
13    LoG.append((sigmas[i]**2)*abs(Lxx+Lyy))
14
15 # For each scale, keep only corners that are maximum in area of 2 scales
16 for i in range(N):

```

```

17 for y, x in zip(*np.where(image_blobs[i])):
18     bool1 = (LoG[i][y, x] > LoG[i-1][y, x]) if i>0 else True #if-else ensures that index stays
    within range
19     bool2 = (LoG[i][y, x] > LoG[i+1][y, x]) if i<len(LoG)-1 else True
20     image_blobs[i][y, x] = bool1 and bool2
21
22 # Show result
23 kp_data = []
24 for i in range(N):
25     for y, x in zip(*np.where(image_blobs[i])):
26         kp_data.append([x, y, sigmas[i]])
27
28     utils.interest_points_visualization(image2_rgb, kp_data)
29

```



Σχήμα 15: Πολυκλιμακωτό blob detection με χρήση box filters